

Generative AI

Exam Preparation Worksheet (2025/26)

Miguel Cabral Pinto & amigos

Made with the context of the entire GenAI course from MEI @ UC. Questions marked ★ are actual exam questions; support project-related claims with experimental observations. Questions shown in grey are optional — they cover niche detail unlikely to appear in a 2-hour exam; do the black ones first.

Contents

Model Family Quick Reference	3
1 Generative AI Foundations	4
2 Classical Generative Methods	5
3 Autoencoders	7
4 Variational Autoencoders	8
5 Generative Adversarial Networks	10
6 Diffusion Models	12
7 Evaluation	14
8 Latent Space Navigation	16
9 LLMs as Sequence Generators	18
10 Modern Image Generation	20
Answer Key	22

Key Equations Reference

Variational Autoencoder

$$\mathcal{L}_{\text{ELBO}} = \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{reconstruction}} - \underbrace{\text{KL}(q_\phi(z|x) \parallel p(z))}_{\text{regularisation}} \quad (1)$$

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I) \quad (\text{reparameterisation trick}) \quad (2)$$

KL decomposition for diagonal Gaussian posteriors:

$$-\frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1)$$

Generative Adversarial Networks

$$\mathcal{L}_{\text{GAN}} = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))] \quad (3)$$

$$\mathcal{L}_{\text{WGAN}} = \mathbb{E}[f(x)] - \mathbb{E}[f(G(z))], \quad f \text{ is 1-Lipschitz} \quad (4)$$

$$\mathcal{L}_{\text{GP}} = \lambda \mathbb{E}_{\hat{x}} [(\|\nabla_{\hat{x}} f(\hat{x})\|_2 - 1)^2], \quad \hat{x} = tx + (1-t)G(z) \quad (\text{WGAN-GP}) \quad (5)$$

DDPM

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I), \quad \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s) \quad (\text{one-shot forward}) \quad (6)$$

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, x_0, \varepsilon} [\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2] \quad (7)$$

$$\hat{\varepsilon}_{\text{cfg}} = \varepsilon_\theta(x_t, t, \emptyset) + w (\varepsilon_\theta(x_t, t, c) - \varepsilon_\theta(x_t, t, \emptyset)) \quad (\text{classifier-free guidance}) \quad (8)$$

Rectified Flow / Flow Matching

$$x_t = (1-t)x_0 + tx_1, \quad x_1 \sim \mathcal{N}(0, I) \quad (\text{straight-line interpolant}) \quad (9)$$

$$\mathcal{L}_{\text{RF}} = \mathbb{E}[\|v_\theta(x_t, t) - (x_1 - x_0)\|^2] \quad (10)$$

β -VAE

$$\mathcal{L}_{\beta\text{-VAE}} = \mathbb{E}[-\log p_\theta(x|z)] + \beta \cdot \text{KL}(q_\phi(z|x) \parallel p(z)), \quad \beta > 1$$

LoRA

$$W' = W + \Delta W = W + BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k)$$

Temperature Sampling

$$P_\tau(x_t = w_j | x_{<t}) = \frac{\exp(z_j/\tau)}{\sum_k \exp(z_k/\tau)}$$

Model Family Quick Reference

	VAE	VQ-VAE	GAN	DDPM	Flow (RF)
Training obj.	ELBO (MSE + KL)	Recon + codebook + commit	Minimax / Wasserstein	$\ \varepsilon - \varepsilon_\theta\ ^2$	$\ v_\theta - (x_1 - x_0)\ ^2$
Latent space	Continuous, $\mathcal{N}(0, I)$	Discrete codebook	Implicit ($z \sim \mathcal{N}$)	Markov noisy chain	Straight ODE
Sample quality	Blurry	High (with prior)	Sharp	High	High
Mode coverage	Good	Good	Poor (collapse risk)	Excellent	Excellent
Speed	Fast	Fast + prior	Fast	Slow (1000 steps)	Few steps
Likelihood	Approx (ELBO)	None	None	Approx	None

1 Generative AI Foundations

Generative AI encompasses any system that constructs novel instances of a target domain. This section establishes the formal distinction between generative and discriminative models, surveys the four main paradigms (symbolic, statistical, evolutionary, connectionist), and examines the broader capabilities generative models provide beyond sample synthesis.

1.1 Which of the following best describes a generative model?

- A) A model that learns a decision boundary $P(Y | X)$ to classify inputs.
- B) A model that learns the data distribution $P(X)$ and can sample new instances from it.
- C) A model that retrieves the nearest stored example from a memory bank.
- D) A model that maps inputs to fixed-length feature vectors for downstream classifiers.

1.2 Evaluate the following claim and explain your reasoning.

“A generative model can always be used as a discriminative classifier via Bayes’ rule, but a discriminative model cannot be used to generate new samples.”

1.3 Complete the sentence:

“The fundamental problem of generative ML is: given examples from some unknown distribution _____, learn to generate new samples that could plausibly have come from _____.”

1.4 Describe the “generative paradox.” Give one concrete example of a model family that resolves each tension it describes.

1.5 Match each approach to its paradigm. Write the number in the blank.

Approach	Paradigm
a AARON (Harold Cohen)	_____
b Gaussian Mixture Models	_____
c L-systems	_____
d GANs	_____
e Genetic algorithms	_____

Paradigms: (1) Statistical (2) Symbolic (3) Evolutionary / ALife (4) Connectionist / Deep

1.6 Explain the difference between “Generative AI” (broad) and “Generative ML” (narrow) as defined in the course. Give one example of a generative AI system that is not a generative ML system.

1.7 List three distinct capabilities that a generative model provides beyond sample synthesis (e.g., data augmentation). For each, give a concrete application domain.

2 Classical Generative Methods

Before deep learning, generative systems were built from explicit rules, evolutionary search, or statistical models over sequences. This section covers symbolic rule systems (AARON, shape grammars, L-systems), constraint-based methods (CSP/CLP), evolutionary approaches (genetic algorithms, biomorphs), and Markov-chain methods for music (EMI, Illiac Suite). Comparing paradigms across controllability, capacity for surprise, and scalability is a recurring exam theme.

2.1 Which statement best describes how AARON generates paintings?

- A) It trains on images and learns to reconstruct them via backpropagation.
- B) It applies stochastic rule selection from a hierarchical symbolic knowledge tree hand-crafted by Harold Cohen.
- C) It uses a genetic algorithm to evolve compositions toward a fitness function.
- D) It samples from a Markov chain trained on Cohen's past paintings.

2.2 Cohen's stated position throughout most of his career was: "AARON is creative; I am not the artist." Evaluate this claim.

2.3 Complete the sentence:

"An L-system differs from a context-free grammar in that rules are applied _____ (simultaneously / one at a time). This models _____ growth, where every cell divides at the same time."

2.4 Describe the generate-and-test approach used in the Illiac Suite. What are the two components and how do they interact?

2.5 Compare AARON (symbolic) and Evolved Virtual Creatures (evolutionary) on four dimensions: source of knowledge, degree of controllability, capacity for surprise, and role of the human designer.

2.6 Shannon's text experiments (1948) showed that increasing the order of a Markov model makes text more locally coherent. State one limitation of high-order Markov models that motivated the move to neural approaches.

2.7 Fill in the missing cells.

Dimension	Symbolic	Evolutionary	Statistical
Knowledge source	Human engineering	_____	Data (learning)
Controllability	High	_____	Medium
Novelty / Surprise	_____	High	Medium
Interpretability	_____	Low	Medium
Scalability	_____	Medium	Medium→High

2.8 (opt.) Explain the role of turtle graphics interpretation in L-systems. Give the axiom and two derivation steps of the algae example ($A \rightarrow AB$, $B \rightarrow A$, axiom A).

2.9 (*opt.*) In a Constraint Satisfaction Problem (CSP) used for generative purposes, what does the “generative space” equal?

- A) The set of all variable assignments tried before a solution is found.
- B) The set of all valid variable assignments satisfying all constraints.
- C) The search heuristic used by the solver.
- D) The number of constraints minus the number of variables.

2.10 ★ Harold Cohen’s AARON and evolutionary art systems (e.g., Karl Sims’ Evolved Virtual Creatures) represent two different philosophies for building generative systems. Compare these classical approaches in terms of the source of “creativity,” their capacity for surprise, and the role of the human designer.

3 Autoencoders

An autoencoder maps inputs through a bottleneck and reconstructs them, forcing the network to learn compact representations. This section covers the classical undercomplete autoencoder and its regularised variants: denoising (input corruption), sparse (activation constraints), contractive (Jacobian penalty), and the convolutional U-Net with skip connections for dense prediction.

3.1 What is the primary reason classical (undercomplete) autoencoders are not suitable as standalone generative models?

- A) They use MSE loss, which is not differentiable.
- B) Their latent space is discontinuous and unstructured, so sampling from it produces garbage.
- C) They cannot be trained with gradient descent.
- D) They always overfit to noise.

3.2 Complete the sentence:

“A denoising autoencoder is trained by: (1) _____ the input with Gaussian noise or masking; (2) encoding the corrupted version; (3) decoding; and (4) computing the loss against the _____ input.”

3.3 Evaluate the following claim.

“A sparse autoencoder must have fewer hidden units than the input dimension.”

3.4 Explain the purpose of skip connections in a U-Net. In what sense does pooling “lose information,” and how do skip connections recover it?

3.5 Compare denoising, sparse, and contractive autoencoders on: (a) how regularisation is applied, (b) what the latent space looks like, and (c) one application each is well suited for.

3.6 Hinton & Salakhutdinov (2006) showed that deep autoencoders outperform PCA on 20Newsgroups for dimensionality reduction. What is the key structural advantage of a deep autoencoder over PCA?

- A) PCA cannot handle more than 100 dimensions.
- B) Autoencoders can learn nonlinear manifolds; PCA is restricted to linear projections.
- C) PCA requires labelled data; autoencoders do not.
- D) Autoencoders use cross-entropy loss; PCA uses Euclidean distance.

3.7 (opt.) Why are transposed convolutions sometimes replaced by nearest-neighbour upsampling followed by a regular convolution in decoder architectures? What artifact does transposed convolution introduce?

3.8 (opt.) Explain the Jacobian penalty in contractive autoencoders. What geometric property does it enforce on the mapping $f : x \mapsto z$?

4 Variational Autoencoders

Variational Autoencoders add a probabilistic prior to the latent space, converting the autoencoder into a proper generative model. This section derives the ELBO objective, explains why the marginal $p_\theta(x)$ is intractable, covers the reparameterisation trick, and examines key failure modes (posterior collapse) and extensions (β -VAE for disentanglement, VQ-VAE for discrete codes).

4.1 In a VAE, the encoder outputs:

- A) A single deterministic latent vector z .
- B) The parameters μ and σ of a Gaussian distribution over z .
- C) A discrete codebook index.
- D) A probability over all training images.

4.2 Complete the sentence:

“The ELBO has two terms. The first term, $\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]$, encourages _____. The second term, $-\text{KL}(q_\phi(z|x) \parallel p(z))$, penalises _____. Maximising the ELBO also pushes _____ up.”

4.3 The VAE cannot maximise $\log p_\theta(x)$ directly because

$$p_\theta(x) = \int p_\theta(x|z) p(z) \, dz.$$

Explain why this integral is intractable.

4.4 What problem does the reparameterisation trick solve?

4.5 Describe “posterior collapse” in a VAE. Which term in the ELBO causes it, and what practical technique (with a one-sentence explanation) is used to avoid it?

4.6 Explain what β -VAE adds to the standard VAE objective and what disentanglement means geometrically. Why does increasing β improve disentanglement at the cost of reconstruction quality?

4.7 Evaluate the following claim.

“Sampling from $\mathcal{N}(0, I)$ and decoding always produces a realistic image in a well-trained classical autoencoder.”

4.8 VQ-VAE avoids posterior collapse because:

- A) It uses a higher learning rate than standard VAEs.
- B) The encoder must commit to a specific discrete codebook entry, so the decoder cannot ignore the latent code.
- C) It uses the Wasserstein distance instead of KL divergence.

D) It trains the encoder and decoder separately.

4.9 Compare VAE and VQ-VAE on: (a) latent space type, (b) susceptibility to posterior collapse, (c) how new samples are generated, and (d) one downstream application enabled by each.

4.10 (*opt.*) Describe the three separate effects of the KL term on the encoder's posterior. What does each of the components $-\mu_j^2$, $-\sigma_j^2$, and $\log \sigma_j^2$ do individually?

4.11 (*opt.*) Explain the straight-through gradient estimator used in VQ-VAE. What problem does it solve and what approximation does it make?

4.12 ★ A Variational Autoencoder is trained by optimising a loss with two main terms. Explain the role of each term and what happens to the generated outputs when one dominates the other.

5 Generative Adversarial Networks

Generative Adversarial Networks frame generation as a two-player minimax game between a generator G and a discriminator D . This section covers the minimax objective and its connection to JS divergence, the main failure modes (mode collapse, vanishing gradients), and the major architectural developments: DCGAN (stable training rules), WGAN/WGAN-GP (Lipschitz constraints), CycleGAN (unpaired translation), and StyleGAN ($\mathcal{W}$ space and AdaIN).

5.1 Complete the sentence:

The GAN minimax objective is:

$$\min_G \max_D \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

At equilibrium, $D(x) = \underline{\hspace{2cm}}$ for all x , meaning the discriminator $\underline{\hspace{2cm}}$.

5.2 Explain why the original generator loss $\min_G \log(1 - D(G(z)))$ has vanishing gradients early in training. What is the non-saturating alternative and why does it help?

5.3 Minimising the GAN objective (with the optimal discriminator substituted in) is equivalent to minimising which divergence between p_{data} and p_g ?

- A) KL divergence
- B) Total variation distance
- C) Jensen–Shannon divergence
- D) Wasserstein-1 distance

5.4 Describe mode collapse precisely. Give one concrete example (e.g., MNIST). Then list three techniques that mitigate it.

5.5 Complete the sentence:

“DCGAN’s key design rules: replace pooling with $\underline{\hspace{2cm}}$ convolutions; use $\underline{\hspace{2cm}}$ normalisation in both G and D ; remove $\underline{\hspace{2cm}}$ layers; use ReLU in G and $\underline{\hspace{2cm}}$ in D .”

5.6 Evaluate the following claim.

“WGAN replaces the discriminator with a critic that outputs a probability between 0 and 1.”

5.7 Explain the Lipschitz constraint in WGAN and why it is necessary. Describe the difference between weight clipping (WGAN) and gradient penalty (WGAN-GP) as two ways of enforcing it.

5.8 Describe the CycleGAN setup. How many networks are involved? What is the cycle consistency loss and why does it prevent mode collapse in the unpaired translation setting?

5.9 Compare Pix2Pix and CycleGAN on: (a) data requirements (paired vs unpaired), (b) number of networks, (c) loss function components, and (d) a typical use case for each.

5.10 StyleGAN introduces a mapping network and an intermediate latent space $\mathcal{W}$. Explain what the mapping network does and why $\mathcal{W}$ is considered more disentangled than the input $\mathcal{Z}$ space. Why does this distinction matter when using StyleGAN for semantic image editing?

5.11 (*opt.*) In BigGAN, the “truncation trick” works by:

- A) Removing the top- k least likely samples at inference.
- B) Interpolating the latent input toward the mean, reducing variance and trading diversity for fidelity.
- C) Truncating the discriminator’s gradient updates to prevent overfitting.
- D) Quantising the latent space to a discrete codebook.

5.12 ★ Explain the adversarial training dynamic of GANs and describe mode collapse as a failure mode, including how you would detect it. You may reference the objective:

$$\mathcal{L}_{\text{GAN}} = \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$$

6 Diffusion Models

Diffusion models learn to reverse a gradual noising process, enabling high-quality generation through iterative denoising. This section covers DDPM's forward and reverse processes, the noise-prediction training objective, DDIM as a deterministic fast-sampling alternative, the two main conditioning schemes (classifier guidance and classifier-free guidance), and latent diffusion for computational efficiency.

6.1 Complete the sentence:

In DDPM's forward process, the noisy image at timestep t can be obtained in one shot as:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

where $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ and $\alpha_s = 1 - \beta_s$. At $t = T$, $\bar{\alpha}_T \approx$ _____, so $x_T \approx$ _____.

6.2 Explain conceptually why predicting the noise ε added to x_0 (rather than predicting μ directly) is both mathematically equivalent and empirically more stable in DDPM.

6.3 Evaluate the following claim.

“DDPM training requires solving a minimax game between two networks, making it as unstable as GAN training.”

6.4 Describe the key difference between DDPM sampling and DDIM sampling. What property of DDIM enables (a) deterministic generation from a fixed noise map and (b) a 10–50× speedup at inference?

6.5 Explain classifier guidance (Dhariwal & Nichol, 2021). Write the guided noise prediction formula and explain what the guidance scale s controls.

6.6 Explain classifier-free guidance (CFG). What training trick enables the model to learn both conditional and unconditional denoising from a single network? Write the CFG formula and explain what happens when $w > 1$.

6.7 Compare classifier guidance and classifier-free guidance on: (a) extra components needed, (b) scalability to text conditioning, and (c) typical guidance scale values in practice.

6.8 Latent Diffusion Models (LDM) run the diffusion process in the latent space of a pretrained autoencoder. Explain the two-stage architecture, the approximate compression ratio for a 512×512 image, and why this reduces compute.

6.9 Imagen (Google Brain, 2022) found that the most important architectural choice for text–image alignment was:

- A) Using a U-Net with more parameters.
- B) Replacing the VAE with a VQ-VAE.

- C) Using a large language model (T5-XXL) as the text encoder.
- D) Training on a larger image dataset.

6.10 Compare VAE, GAN, and DDPM across: training stability, sample quality, mode coverage, generation speed, and whether an explicit likelihood is available.

6.11 (*opt.*) The U-Net used in DDPM is conditioned on the timestep t . How is t injected into the network?

- A) As an additional input channel concatenated to the noisy image.
- B) Via sinusoidal positional embeddings processed through FiLM conditioning layers.
- C) As a scalar multiplied by the final output layer's weights.
- D) Through cross-attention with a learned t -specific token.

6.12 ★ Describe the forward and reverse processes in DDPMs at a conceptual level. Then explain classifier-free guidance both in terms of mechanism and impact on image generation.

7 Evaluation

Evaluating generative models is non-trivial: quality is multi-dimensional and there is no single ground-truth metric. This section covers the main distributional metrics (FID, KID, IS), their underlying assumptions and failure modes, complementary metrics (Precision/Recall, LPIPS, CLIP-Score, CleanFID), and the experimental protocol choices that affect reproducibility and cross-paper comparability.

7.1 Complete the sentence:

“FID computes the Fréchet distance between real and generated sample distributions in the feature space of a pretrained _____ network. Lower FID means _____ quality and _____ diversity.”

7.2 Explain the Gaussian assumption embedded in FID. Name one scenario where this assumption is seriously violated and how it affects the validity of FID scores.

7.3 Evaluate the following claim.

“A model with a very low FID is necessarily diverse.”

7.4 Explain the conceptual difference between Precision and Recall for generative models (Kynkäänniemi et al.). Give an example of a failure mode that Precision catches but FID misses, and one that Recall catches but FID misses.

7.5 KID differs from FID in that:

- A) KID uses a different pretrained feature extractor.
- B) KID makes no Gaussian assumption and provides an unbiased estimator, making it more reliable at small sample counts.
- C) KID measures per-sample quality rather than distributional similarity.
- D) KID uses pixel space rather than feature space.

7.6 List three limitations of CLIPScore as a metric for text-to-image models. What metric would you pair it with for a more complete evaluation?

7.7 Explain the protocol design problem in evaluation. Name four experimental choices that, if reported differently, can make the same metric value scientifically meaningless.

7.8 Compare Inception Score (IS) and FID on: (a) whether they reference real data, (b) what each measures, and (c) one fundamental limitation of each.

7.9 (opt.) Explain the “CleanFID” problem. What is “implementation drift” and why can two papers report FID on the same model and reach contradictory conclusions?

7.10 ★ The project used FID and KID as the primary evaluation metrics. Critically evaluate this protocol: what aspects of generation quality do FID and KID capture well, what do they miss, and how could the protocol be misleading? If you could add one additional evaluation method to better understand the differences between your three models, what would it be and why?

8 Latent Space Navigation

The latent spaces of trained generative models contain structured semantic directions that can be found and traversed without retraining. This section covers interpolation methods (lerp, slerp), unsupervised direction discovery (GANSpace, SeFa), supervised direction discovery (InterFaceGAN), GAN inversion (Image2StyleGAN, pSp, e4e), and diffusion-era editing tools (Prompt-to-Prompt, Null-Text Inversion, ControlNet, DreamBooth).

8.1 Complete the sentence:

“Linear interpolation between two latent codes is $z(t) = (1 - t)z_0 + tz_1$. Spherical interpolation (slerp) is preferred when the prior is _____ because it follows the _____ between the two points, staying in higher-density regions.”

8.2 Explain the fundamental reconstruction–editability trade-off in GAN inversion. Why does optimising aggressively for reconstruction push z^* into a region of low editability?

8.3 Explain how Prompt-to-Prompt edits images. What is “swapped” during the denoising process, and how does this preserve spatial layout while changing semantic content?

8.4 Evaluate the following claim.

“DDIM inversion works perfectly even with high classifier-free guidance weights.”

8.5 Explain the ControlNet “clone trick”: what is frozen, what is trainable, and why do zero convolutions guarantee that base model quality is never degraded during early training?

8.6 Compare Textual Inversion and DreamBooth on: (a) what parameters are updated, (b) training images required, (c) output file size, and (d) typical fidelity to the target concept.

8.7 Explain the two main families of approach for finding semantic editing directions in a GAN’s latent space: supervised (label-based) and unsupervised (data-driven). What information does each require, and what is the key trade-off between them? Give one example of each approach.

8.8 (opt.) GANSpace discovers editing directions by:

- A) Training a linear SVM on attribute labels in $\mathcal{W}$ space.
- B) Applying PCA to the intermediate activation space of the StyleGAN generator, finding principal axes of variation.
- C) Computing the eigenvectors of the first generator layer’s weight matrix $AA^\top$.
- D) Minimising a CLIP similarity loss over the latent space.

8.9 (opt.) Explain SeFa (Semantic Factorisation). What mathematical object is factorised, and why is this “architecture-intrinsic” rather than “empirically discovered”?

8.10 (*opt.*) Compare Image2StyleGAN, pSp, and e4e on: (a) inversion approach (optimisation vs encoder), (b) inference speed, (c) reconstruction quality, and (d) editability of the resulting code.

8.11 (*opt.*) Explain Null-Text Inversion. What two stages does it run, and what is the null embedding optimised to do?

9 LLMs as Sequence Generators

Large language models generate text autoregressively, one token at a time, by learning $P(x_t | x_{<t})$ over a large corpus. This section covers the autoregressive factorisation, subword tokenisation (BPE), decoding strategies and their quality–diversity trade-offs, prompting paradigms (zero-shot, few-shot, chain-of-thought), hallucinations, and the scaling laws that govern LLM performance.

9.1 Complete the sentence:

“The autoregressive factorisation decomposes a sequence probability as $P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t | x_{<t})$. This is _____ (exact / approximate). GPT-style models learn each conditional by training with _____ prediction on a large corpus.”

9.2 Top- p (nucleus) sampling selects the smallest set of tokens whose cumulative probability is $\geq p$. Its key advantage over top- k sampling is:

- A) It is faster to compute.
- B) The set size adapts dynamically: small when the model is confident, large when it is uncertain.
- C) It always selects exactly k tokens.
- D) It eliminates the need for temperature.

9.3 Describe the effect of temperature τ on the output distribution. What happens as $\tau \rightarrow 0$ and as $\tau \rightarrow \infty$? Give the formula for the temperature-scaled distribution.

9.4 Evaluate the following claim.

“Beam search always produces the globally highest-probability sequence.”

9.5 Explain the concept of “hallucination” in LLMs. Distinguish between factual hallucination and faithfulness hallucination. Why does the autoregressive training objective not explicitly discourage hallucination?

9.6 Explain why chain-of-thought (CoT) prompting improves performance on multi-step reasoning tasks. Describe the mechanism in terms of how intermediate tokens become part of the model’s context.

9.7 Compare zero-shot, few-shot, and in-context learning as control mechanisms for LLMs. For each, describe how the model’s behaviour is shaped and what assumption it relies on.

9.8 Explain how subword tokenisation (BPE) works at a high level and why it is preferred over word-level or character-level tokenisation for LLMs.

9.9 (opt.) The Chinchilla result (Hoffmann et al., 2022) showed that:

- A) Larger models always outperform smaller models regardless of data.

- B) For a fixed compute budget, many early LLMs were over-parameterised and under-trained on data, and performance is jointly determined by model size and dataset size.
- C) The optimal strategy is to train on as much data as possible at constant model size.
- D) Model size follows a power law with dataset size only.

9.10 ★ Autoregressive language models generate text one token at a time, and the sampling strategy strongly affects the output. Compare temperature scaling, top- k , and nucleus (top- p) sampling as three ways of controlling the balance between diversity and coherence.

10 Modern Image Generation

Recent advances have moved beyond the convolutional U-Net backbone and the DDPM training objective. This section covers transformer-based diffusion (DiT, MMDiT), flow matching and rectified flow as cleaner training objectives, single-step generation via distillation (consistency models, ADD), the FLUX/SD3 architecture family, fine-tuning methods (LoRA, DreamBooth), and the autoregressive revival enabled by better visual tokenisers.

10.1 DiT (Diffusion Transformer, Peebles & Xie 2023) differs from the U-Net backbone in that:

- A) It uses a completely different training loss.
- B) It replaces the convolutional encoder-decoder with a transformer operating on latent patches, enabling LLM-style scaling laws.
- C) It removes the VAE and works in pixel space.
- D) It uses adversarial training instead of denoising.

10.2 Explain rectified flow as a training objective. Write the interpolant, the target velocity, and the training loss. Why do straight-line trajectories allow fewer ODE integration steps at inference compared to DDPM?

10.3 Evaluate the following claim.

“DDPM is a special case of the flow matching framework.”

10.4 Explain consistency models. What is the “consistency property” $f_\theta(x_t, t) = x_0$ and how does it enable one-step generation? Distinguish between Consistency Training (CT) and Consistency Distillation (CD).

10.5 LoRA (Low-Rank Adaptation) works by:

- A) Replacing all weight matrices with their SVD decomposition.
- B) Freezing the base model and adding a trainable low-rank decomposition $\Delta W = BA$ alongside each weight matrix.
- C) Fine-tuning only the normalisation layers while freezing everything else.
- D) Distilling the base model into a smaller student network.

10.6 Explain the DreamBooth fine-tuning approach. What is the “class-preservation loss” and why is it needed?

10.7 (opt.) Complete the sentence:

“In adaLN-Zero, the conditioning signals (timestep t , class label, text) are used to predict _____ and _____ that modulate the LayerNorm statistics of each transformer block. An additional scalar _____, initialised to _____, gates each residual branch so that at initialisation every block is an _____ function.”

10.8 (*opt.*) Explain Adversarial Diffusion Distillation (ADD), used in SDXL-Turbo. How does combining a distillation loss with a GAN discriminator loss address the diversity gap left by standard distillation?

10.9 (*opt.*) Compare Stable Diffusion 3 / FLUX.1 and DALL-E 2 on: (a) architecture backbone, (b) training objective, (c) text encoder, and (d) conditioning mechanism.

10.10 (*opt.*) Explain the MMDiT (Multi-Modal Diffusion Transformer) block used in SD3 and FLUX. How does it differ from a standard DiT block in how text and image tokens interact?

10.11 (*opt.*) Compare the autoregressive revival (VAR, MAR, Emu3) against diffusion-based image generation on: (a) generation speed, (b) architectural unification with text models, (c) sample quality on benchmarks, and (d) the key enabler that made AR competitive again.

10.12 ★ Suppose you are asked to repeat the project at 256×256 resolution instead of 32×32 . For each model family (VAE, DCGAN, DDPM), describe what would break or need to change architecturally. Which family benefits most from the increased resolution, and would you replace any of the three with a different approach at this scale? Justify your answer based on what you learned from the project.

Answer Key

Section 1 — Generative AI Foundations

1.1 B.

1.2 True. A generative model $p(x, y)$ can recover $P(y|x) = p(x, y)/p(x)$ via Bayes' rule, so classification is always possible from a generative model. A discriminative model learns only $P(y|x)$ and has no representation of $p(x)$, so it cannot generate new x samples.

1.3 "...from some unknown distribution $p(x)$...from the same distribution."

1.4 The generative paradox is the set of opposing tensions a generative model must navigate simultaneously: learn patterns without memorising data; be coherent without becoming repetitive; be diverse without losing structure; be realistic without overfitting; be novel without becoming random. VAEs resolve the memorisation/smoothness tension via KL regularisation; GANs address the realism/blur tension via adversarial loss; diffusion models handle the quality/diversity tension via iterative denoising.

1.5 a-2 (Symbolic), b-1 (Statistical), c-3 (Evolutionary/ALife), d-4 (Connectionist/Deep), e-3 (Evolutionary/ALife).

1.6 Generative AI (broad) includes any system that constructs novel artifacts from a target domain—rule-based systems, evolutionary algorithms, CSP solvers—none of which learn from data. Generative ML (narrow) specifically learns a distribution $p(x)$ from examples and samples from it. Example of non-ML generative AI: AARON, which generates paintings via hand-crafted symbolic rules without any data-driven learning.

1.7 (1) Anomaly detection via likelihood scoring—used in fraud detection. (2) Missing data imputation by sampling plausible completions—used in medical imaging. (3) Data augmentation to balance class distributions—used in rare-disease classification.

Section 2 — Classical Generative Methods

2.1 B.

2.2 False (contested). Cohen's original position was that he was the artist and AARON was a sophisticated tool. Over time he reconsidered and argued that AARON exhibited a form of creativity, since its outputs surprised even him. The question of where creativity "lives" remains unresolved; the answer depends on whether combinatorial novelty within a hand-coded rule space constitutes creativity.

2.3 Simultaneously; biological.

2.4 The Illiac Suite uses a generate-then-test loop: a stochastic process (random note generation) proposes candidates, and a rule-based screener applies counterpoint constraints to accept or reject each. Randomness provides exploration; rules enforce musical validity.

2.5

Dimension	AARON (Symbolic)	Evolved Virtual Creatures
Knowledge source	Hand-coded by Cohen over 40+ years	Emergent from fitness-driven search
Controllability	High (rules are explicit)	Low-medium (fitness shapes, not specifies)
Capacity for surprise	Low (combinations of known rules)	High (alien morphologies the designer never imagined)
Human designer role	Author of all aesthetic knowledge	Designer of fitness function and genotype representation

2.6 The state space grows exponentially with order n : a k -gram model over vocabulary V has V^{k-1} possible contexts, so most transitions are never observed in training data, causing data sparsity and unreliable estimates. RNNs learn a compressed hidden state rather than enumerating all contexts, removing this bottleneck.

2.7 Evolutionary column: Emergent (search); Low–Medium. Symbolic column: Low (novelty); High (interpretability); Low (scalability).

2.8 Turtle graphics assigns a physical interpretation to the L-system string: F moves forward drawing a line, + turns right, – turns left, [saves state,] restores state. Algae example: $n = 0$: A; $n = 1$: AB (apply $A \rightarrow AB$); $n = 2$: ABA (apply $A \rightarrow AB$ to A, $B \rightarrow A$ to B, simultaneously).

2.9 B.

2.10 AARON uses hierarchical production rules encoding all of Cohen’s aesthetic knowledge. Every output is a combination of pre-existing rules; novelty comes from the combinatorial space of rule interactions, not from learning or optimisation. The human designer is the sole source of all creativity, and no output can exceed the encoded knowledge. Evolutionary art systems (e.g., Sims) use a fitness function and genetic operators; no behavioural knowledge is hand-coded. Variation comes from mutation and crossover; fitness selection drives adaptation. The designer specifies the fitness landscape and representation, but the system produces morphologies and behaviours the designer never imagined. Key contrast: AARON’s creativity is bounded by Cohen’s knowledge; evolutionary systems generate novelty by search and can surprise their designers in ways AARON, by design, cannot.

Section 3 — Autoencoders

3.1 B.

3.2 Corrupting; clean.

3.3 False. A sparse autoencoder is typically overcomplete ($\dim(z) \geq \dim(x)$). Sparsity is enforced not by reducing neurons but by constraining how many are active at once (via L1 penalty, KL divergence on activation rates, or top- k selection). This allows a richer vocabulary of features than a small bottleneck would.

3.4 Pooling progressively reduces spatial resolution, discarding fine-grained spatial detail in favour of higher-level features. Skip connections in U-Net concatenate the encoder’s feature maps (before pooling) directly to the corresponding decoder layers, giving the decoder access to high-resolution spatial information that pooling would otherwise destroy. This is essential for dense prediction tasks where output resolution must match input resolution.

3.5

	Denoising AE	Sparse AE	Contractive AE
Regularisation	Corrupts input; loss on clean reconstruction	Penalises active units (L1/KL/top- k)	Penalises Jacobian Frobenius norm
Latent space	Robust to noise perturbations	High-dimensional, only few dims active	Locally invariant; contracts input neighbourhoods
Application	Image/audio denoising; BERT pretraining	Interpretable feature learning	Robust representations for noisy sensors

3.6 B.

3.7 Transposed convolutions with kernel size not divisible by stride cause some output positions to receive contributions from more input values than others, producing a periodic “checkerboard” artifact. Replacing them with bilinear or nearest-neighbour upsampling followed by a regular convolution avoids this because the upsampling step has no learnable parameters and the subsequent convolution smooths the result uniformly.

3.8 The CAE adds a penalty $\lambda \|J_f(x)\|_F^2$ on the Frobenius norm of the encoder’s Jacobian $J_f = \partial f(x)/\partial x$. Small Frobenius norm means the encoder’s output changes very little when the input is perturbed, so the encoder contracts local volumes in input space to smaller volumes in latent space. This enforces

local invariance: representations are stable to small nuisances while still discriminating between genuinely different inputs.

Section 4 — Variational Autoencoders

4.1 B.

4.2 Reconstruction quality; deviation of the encoder’s posterior from the prior $\mathcal{N}(0, I)$; $\log p_\theta(x)$.

4.3 The integral over $z \in \mathbb{R}^d$ is intractable because the decoder is a neural network with no closed-form marginal and d is typically hundreds of dimensions, so direct integration is computationally impossible. The reparameterisation trick solves the training problem: writing $z = \mu + \sigma \odot \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I)$ moves the stochastic node to ε , which has no learnable parameters, so gradients flow through μ and σ via backpropagation.

4.4 The sampling step $z \sim q_\phi(z|x)$ is stochastic and non-differentiable, so gradients cannot flow through it to update ϕ . The reparameterisation trick rewrites $z = \mu_\phi(x) + \sigma_\phi(x) \odot \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I)$; the randomness moves to ε (no parameters), and gradients now flow through μ_ϕ and σ_ϕ normally.

4.5 Posterior collapse occurs when the KL term dominates training, pushing the encoder’s posterior toward $\mathcal{N}(0, I)$ regardless of input so the decoder ignores z entirely and generates average outputs. KL annealing avoids this by starting training with the KL weight at zero and linearly increasing it over the first epochs, letting the encoder learn meaningful codes before regularisation pressure builds.

4.6 β -VAE multiplies the KL term by $\beta > 1$: $\mathcal{L} = \mathbb{E}[-\log p_\theta(x|z)] + \beta \cdot \text{KL}(q_\phi(z|x)||p(z))$. Disentanglement means each latent dimension captures a single independent generative factor. Increasing β pushes q_ϕ harder toward the isotropic Gaussian prior, encouraging independence between dimensions since the prior factorises. The cost is that the reconstruction term carries less weight, so the model cannot use as much latent capacity for reconstruction, producing blurrier outputs.

4.7 False. A classical autoencoder has no defined distribution over its latent space. Decoding a random $z \sim \mathcal{N}(0, I)$ typically produces garbage because the encoder may have mapped inputs to isolated points anywhere in $\mathbb{R}^d$, leaving large “holes” between them with no meaningful decoding.

4.8 B.

4.9

	VAE	VQ-VAE
Latent space	Continuous, $\mathcal{N}(0, I)$ prior	Discrete codebook, spatial grid of tokens
Posterior collapse	Susceptible with expressive decoder	Eliminated: encoder must commit to a discrete code
New samples	Sample $z \sim \mathcal{N}(0, I)$, decode	Need a learned prior (PixelCNN/Transformer), then decode
Downstream use	Smooth interpolation; drug molecule optimisation	DALL-E 1 image tokens; AudioLM speech tokens

4.10 $-\mu_j^2$ pulls latent means toward zero, preventing the encoder from mapping inputs to arbitrarily distant regions. $-\sigma_j^2$ prevents variance from exploding. $\log \sigma_j^2$ prevents variance from collapsing to zero, which would turn distributions into delta functions and eliminate the regularisation benefit.

4.11 The nearest-neighbour lookup $k^* = \arg \min_k \|z_e - e_k\|_2$ is non-differentiable (arg min has zero gradient everywhere). The straight-through estimator pretends in the backward pass that quantisation is an identity: gradients from the reconstruction loss pass directly from z_q back to z_e as if $z_q = z_e$. The approximation holds empirically after the codebook has stabilised and the quantisation error is small.

4.12 Term 1 (reconstruction): $\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]$ pushes the decoder to reconstruct the input accurately from samples of the encoder’s posterior. When this term dominates ($\beta \rightarrow 0$), the model reduces to a classical autoencoder with good reconstruction but an unstructured latent space, so sampling from $\mathcal{N}(0, I)$ hits gaps and produces noise. Term 2 (KL regularisation): $-\text{KL}(q_\phi(z|x)||p(z))$ pulls the encoder’s posterior toward the prior, making the latent space dense and samplable. When this term dominates ($\beta \gg 1$), posterior collapse occurs: the encoder ignores the input and maps everything to the prior, the decoder outputs the training-data mode, and all generated samples are identical.

Section 5 — Generative Adversarial Networks

5.1 0.5; cannot distinguish real from fake.

5.2 When D is confident $G(z)$ is fake, $D(G(z)) \approx 0$, so $\log(1 - D(G(z))) \approx 0$: the loss is flat with near-zero gradient. The non-saturating alternative has G maximise $\log D(G(z))$ instead: when $D(G(z)) \approx 0$ this equals $\log(0) \rightarrow -\infty$, giving a large gradient and a strong learning signal early in training. The fixed point of both objectives is the same; optimisation is substantially more stable with the non-saturating form.

5.3 C.

5.4 Mode collapse: the generator maps many different z values to the same output (or a small set of outputs) that reliably fools D , ignoring the training data’s diversity. On MNIST, a collapsed generator might produce only 3s and 7s. Three mitigation techniques: (1) Wasserstein loss with gradient penalty; (2) minibatch discrimination (discriminator sees groups of samples, penalising low-diversity batches); (3) feature matching (train G to match statistics of real data in D ’s feature space, not just fool D at the output level).

5.5 Strided; batch; fully connected; LeakyReLU.

5.6 False. WGAN’s critic outputs an unbounded real-valued score, not a probability between 0 and 1. The score difference between real and fake samples approximates the Wasserstein-1 distance via the Kantorovich–Rubinstein dual.

5.7 The Kantorovich–Rubinstein dual requires the critic f to be 1-Lipschitz: $|f(x) - f(y)| \leq \|x - y\|$ for all x, y , so that the dual formula holds exactly. Without this, the critic grows without bound. Weight clipping enforces Lipschitzness by clamping all critic weights to $[-c, c]$ after each update, but is crude: it can cause vanishing gradients if c is too small and distorts the critic’s capacity. WGAN-GP penalises the critic when its gradient norm deviates from 1, a smoother and more principled constraint that does not distort network capacity.

5.8 CycleGAN uses four networks: G_{AB} , G_{BA} , D_A , D_B . Cycle consistency requires $G_{BA}(G_{AB}(x)) \approx x$ and $G_{AB}(G_{BA}(y)) \approx y$. Without this constraint, each generator could map all inputs to a single output that fools its discriminator, since there are no paired examples to enforce correctness. The cycle constraint ties the two generators together, preventing either from ignoring input content.

5.9

	Pix2Pix	CycleGAN
Data	Paired (input–output pairs required)	Unpaired (two unrelated image sets)
Networks	2 (1 U-Net G , 1 PatchGAN D)	4 (G_{AB} , G_{BA} , D_A , D_B)
Loss components	Adversarial + L1 reconstruction	Adversarial ($\times 2$) + cycle consistency ($\times 2$)
Typical use case	Semantic map $\rightarrow$ photo; sketch $\rightarrow$ image	Horse $\leftrightarrow$ zebra; summer $\leftrightarrow$ winter

5.10 The mapping network is an 8-layer MLP that maps a sampled $z \in \mathcal{Z} \sim \mathcal{N}(0, I)$ to an intermediate code $w \in \mathcal{W}$. The $\mathcal{Z}$ space must match a fixed Gaussian prior, which forces entangled features together wherever the training distribution requires it. The mapping network can learn a non-linear transformation that unravels this entanglement: different dimensions of w more consistently control single semantic attributes (hair colour, pose, lighting) independently. For editing, this matters because directions in $\mathcal{W}$ more cleanly correspond to individual attributes — moving along a direction in $\mathcal{W}$ changes one feature while leaving others stable, whereas editing in $\mathcal{Z}$ tends to produce correlated, entangled changes.

5.11 B.

5.12 G and D play a minimax game: D maximises $\mathcal{L}_{\text{GAN}}$ by correctly labelling real inputs (output near 1) and fake inputs (output near 0); G minimises by producing samples $G(z)$ that D classifies as real. At Nash equilibrium, $D(x) = 0.5$ everywhere and $p_g = p_{\text{data}}$. Mode collapse: G learns to map many z values to the same output (or a small set) that reliably fools D , ignoring training data diversity. D then adapts to reject those outputs; G shifts to a new mode; the system cycles without converging. Detection: generate a large batch and observe lack of variety; apply a pretrained classifier to generated samples and check if only a few classes appear; monitor the variance of generated samples over training.

Section 6 — Diffusion Models

6.1 0; pure Gaussian noise $\mathcal{N}(0, I)$.

6.2 Predicting ε and predicting μ_θ are mathematically equivalent since the reverse mean is a deterministic function of ε_θ and x_t . Predicting ε is empirically more stable because it is a zero-mean target regardless of timestep, giving the network a consistent regression objective. Predicting μ directly has a scale that changes with $\bar{\alpha}_t$, introducing instability across the 1000-step range.

6.3 False. DDPM training is a standard supervised regression problem: the loss is $\mathbb{E}\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$, a plain MSE with no adversarial game and no competing networks. This is one of the main reasons diffusion models displaced GANs for high-quality generation.

6.4 DDPM uses a Markovian stochastic process adding fresh noise at each step; all 1000 steps are required and outputs differ between runs. DDIM defines a deterministic ODE that shares the same training objective. (a) The same x_T always produces the same image (no stochastic z added), enabling inversion and interpolation in noise space. (b) Because the trajectory is deterministic and smooth, steps can be skipped—running only 50–100 of the 1000—without significant error, since the ODE solver can take larger steps on a smooth function.

6.5 Classifier guidance trains a separate classifier $p(y|x_t)$ on noisy images. At sampling:

$$\hat{\varepsilon}_{\text{guided}} = \varepsilon_\theta(x_t, t) - s\sqrt{1 - \bar{\alpha}_t} \nabla_{x_t} \log p(y | x_t)$$

The gradient pushes x_t toward images the classifier assigns high probability to class y . Scale $s = 0$ is unconditional; larger s increases class adherence at the cost of diversity.

6.6 During training, the condition c is randomly dropped (replaced with $\emptyset$, $\approx 10\%$ of batches), so a single network learns both $\varepsilon_\theta(x_t, t, c)$ and $\varepsilon_\theta(x_t, t, \emptyset)$. At sampling:

$$\hat{\varepsilon}_{\text{cfg}} = \varepsilon_\theta(x_t, t, \emptyset) + w(\varepsilon_\theta(x_t, t, c) - \varepsilon_\theta(x_t, t, \emptyset))$$

When $w = 1$ this is standard conditional generation. When $w > 1$, the guided prediction extrapolates away from the unconditional direction, amplifying the conditioning signal. Higher w improves prompt adherence and sharpness but reduces diversity and can produce oversaturated artifacts.

6.7

	Classifier guidance	Classifier-free guidance
Extra components	Separate noisy-image classifier	None (single network + condition dropout)
Text scalability	Poor (requires a text–image noisy classifier)	Excellent
Typical guidance scale	$s \approx 5\text{--}15$	$w \approx 3\text{--}8$

6.8 Stage 1: a pretrained VAE encodes a $512 \times 512 \times 3$ image to a $64 \times 64 \times 4$ latent ($\approx 48\times$ compression). Stage 2: the U-Net runs the DDPM denoising objective on this compact latent; at generation, the final denoised latent is decoded by the frozen VAE decoder. Compute scales roughly quadratically with spatial resolution, so operating at 64×64 instead of 512×512 gives $(512/64)^2 = 64\times$ fewer spatial tokens, dramatically reducing memory and compute while preserving perceptual quality through the VAE’s learned compression.

6.9 C.

6.10

	VAE	GAN	DDPM
Training stability	Good (ELBO, no game)	Poor (mode collapse, vanishing gradients)	Excellent (MSE regression)
Sample quality	Often blurry	Sharp, realistic	High quality + diverse
Mode coverage	Good	Mode collapse risk	Excellent
Speed	Fast (1 pass)	Fast (1 pass)	Slow (1000 steps; 50 with DDIM)
Explicit likelihood	Approx (ELBO lower bound)	None (implicit)	Approx (simplified ELBO)

6.11 B.

6.12 Forward process: a fixed Markov chain adds Gaussian noise over $T = 1000$ steps via schedule $\{\beta_t\}$; after T steps, the image is indistinguishable from $\mathcal{N}(0, I)$; no learning occurs here. Reverse process: a U-Net conditioned on timestep t learns to predict the noise ε added at each step; generation starts from $x_T \sim \mathcal{N}(0, I)$ and iteratively denoises. CFG mechanism: during training, the condition c is randomly dropped ($\approx 10\%$ of batches), so the network learns both conditional and unconditional denoising from a single model. At sampling, $\hat{\varepsilon} = \varepsilon_\theta(\emptyset) + w(\varepsilon_\theta(c) - \varepsilon_\theta(\emptyset))$ extrapolates toward the conditional direction. Higher w improves prompt alignment and sharpness but reduces diversity and can oversaturate.

Section 7 — Evaluation

7.1 Inception-v3; better; better (FID combines quality and coverage into a single scalar).

7.2 FID models both real and generated feature distributions as multivariate Gaussians. Real image distributions are often multi-modal (images from different classes or art styles), which violates this assumption. With multiple modes, the fitted Gaussian has inflated covariance, making the Fréchet distance less interpretable and potentially rewarding a generated distribution that concentrates on one mode.

7.3 False. A model with severe mode collapse that produces one very high-quality mode can still achieve a low FID if that mode coincidentally matches the mean and covariance of the full real distribution. FID cannot separately measure diversity; Precision/Recall is needed to detect this failure.

7.4 Precision = fraction of generated samples that fall within the real data manifold (measures fidelity). Recall = fraction of the real manifold covered by generated samples (measures diversity). Precision catches mode collapse with sharp images: a model generates high-quality images from a small region, but recall is low—FID may miss this if the covered region matches the overall statistics. Recall catches a model that covers the full distribution with blurry outputs: precision is low, but the broad coverage may keep FID acceptable.

7.5 B.

7.6 (1) CLIPScore does not measure image quality or realism: a blurry or artifact-filled image can score highly if it matches the text description. (2) CLIP has biases from its training distribution and fails on fine-grained compositional reasoning (“a red ball to the left of a blue cube”). (3) It does not penalise spatial errors if the global embedding is similar. Pair with FID to capture distributional quality and diversity.

7.7 Four choices: (1) the image resizing algorithm applied before feature extraction; (2) the version and layer of the feature extractor (Inception-v3 vs CLIP ViT); (3) the number of samples used (FID variance is high below $\approx 10k$ images); (4) the random seed and sampling hyperparameters (temperature, guidance scale) used during generation.

7.8

	Inception Score (IS)	FID
References real data?	No (only classifies generated images)	Yes (compared to real distribution)
Measures	Quality (classifier confidence) + diversity (marginal entropy)	Distance between real and generated distributions in Inception feature space
Fundamental limitation	Can be gamed by training on ImageNet classes; no notion of distributional match	Assumes Gaussians; sensitive to sample count and preprocessing

7.9 FID depends on the image resize algorithm (bilinear, bicubic, Lanczos), JPEG compression decisions, and normalisation choices before feature extraction. Different codebases make different low-level choices, shifting feature distributions and thus FID scores, even on identical images. A model could appear to “improve” FID simply by matching the preprocessing of the reference dataset rather than generating better images.

7.10 What FID and KID capture: both compare real and generated image distributions in Inception feature space, measuring whether the generated distribution matches the real distribution in mean, covariance (FID), and kernel-based statistics (KID); they are sensitive to both quality and diversity. What they miss: (1) neither measures prompt faithfulness; (2) neither measures per-image artifacts not captured by Inception features; (3) FID assumes Gaussian distributions; (4) neither captures fine-grained stylistic differences between art styles. How the protocol can be misleading: high CFG scales improve FID by sharpening samples at the cost of diversity; different preprocessing choices produce incomparable numbers; small sample sets inflate KID variance. Additional method: Precision and Recall (Kynkäänniemi et al.) separates fidelity (Precision) from diversity (Recall), making mode collapse visible independently of overall distributional distance.

Section 8 — Latent Space Navigation

8.1 Spherical (the prior is a unit Gaussian, so points lie roughly on a hypersphere); great-circle arc.

8.2 The $\mathcal{W}$ space of StyleGAN corresponds to codes the training distribution actually used. Optimising the reconstruction loss aggressively pushes z^* into $\mathcal{W}^+$ (per-layer space) to fit fine-grained image details not expressible by the lower-capacity $\mathcal{W}$. Codes far from the training distribution’s mean were never encountered during training, so moving along a semantic direction from such a code produces unpredictable changes rather than the clean semantic edits the direction was calibrated for.

8.3 Prompt-to-Prompt injects the cross-attention maps from the source generation into the corresponding denoising steps of the target generation. The attention map encodes which spatial positions are controlled by which text tokens. Keeping the source maps fixed while changing the text tokens preserves structural layout (each position still “looks at” the same text token position) while changing the semantic content rendered at those positions.

8.4 False. CFG blends conditional and unconditional noise predictions with weight w . The DDIM inversion derivation assumes only the conditional score, so the inversion trajectory diverges from the forward trajectory when $w > 1$. At standard guidance weights ($w \approx 7.5$), accumulated error is large enough that decoding the inverted noise does not faithfully reconstruct the original image.

8.5 ControlNet clones the encoder half of the U-Net (trainable) while keeping the original U-Net frozen. Zero-initialised 1×1 convolutions connect the clone’s output to the frozen decoder. At training step 0, the zero convolutions multiply all clone outputs by zero, so the output is mathematically identical to the frozen base model regardless of what the clone receives. Base model quality can therefore never be degraded during early training, and the gradient landscape starts from the already-trained state.

8.6

	Textual Inversion	DreamBooth
Parameters updated	New token embedding S^* only (~kilobytes)	Full U-Net weights (or LoRA delta)
Training images	3–5	3–5
Output file size	~1 KB	Gigabytes (full) or 50–200 MB (LoRA)
Fidelity	Moderate; style and high-level features	High; identity preserved in fine detail

8.7 Supervised (label-based): train a linear classifier (e.g., SVM) on latent codes paired with binary attribute labels obtained from a pretrained attribute classifier. The decision boundary’s normal vector is the editing direction. Requires labelled attributes but produces semantically targeted, interpretable directions (e.g., “add glasses”, “change age”). Example: InterFaceGAN. Unsupervised (data-driven): discover principal axes of variation by applying PCA to latent codes or intermediate activations, requiring no labels. Directions correspond to the most dominant visual variations but may not map to human-interpretable attributes. Example: GANSpace. Trade-off: supervised gives control over specific named attributes at the cost of needing labelled data; unsupervised gives cheap, label-free exploration of dominant variation axes but with less interpretable semantics.

8.8 B.

8.9 SeFa computes the eigenvectors of $AA^\top$, where A is the weight matrix of the first generator layer. The principal eigenvectors are directions in $\mathcal{W}$ that produce the largest changes in early features, hence the largest semantic variation. This requires no samples and no labels: directions are derived directly from the weight matrix, making the method architecture-intrinsic.

8.10

	Image2StyleGAN	pSp	e4e
Approach	Per-image optimisation in $\mathcal{W}^+$	Encoder (feature pyramid)	Encoder + editability regularisation
Speed	Slow ($\sim 1\text{--}2$ min per image)	Fast (single forward pass)	Fast
Reconstruction quality	High	Good	Slightly lower
Editability	Low (deep $\mathcal{W}^+$ codes)	Moderate	High (regularised toward $\mathcal{W}$)

8.11 Stage 1 (pivot trajectory): standard DDIM inversion on the source image, producing reference noisy latents $\{x_T^*, \dots, x_1^*\}$. Stage 2 (null-text optimisation): at each timestep t , the unconditional embedding $\emptyset_t$ is optimised (model weights frozen) so that CFG-guided denoising from x_t^* exactly reproduces x_{t-1}^* from the pivot. This resolves the CFG-induced drift while preserving editing capability via the unchanged source prompt.

Section 9 — LLMs as Sequence Generators

9.1 Exact; next-token.

9.2 B.

9.3 Temperature τ scales logits before softmax: $P_\tau(x_t = w_j \mid x_{<t}) = \exp(z_j/\tau) / \sum_k \exp(z_k/\tau)$. As $\tau \rightarrow 0$, the distribution peaks at the highest-logit token (greedy decoding, fully deterministic). As $\tau \rightarrow \infty$, the distribution becomes uniform. In practice, $\tau \approx 0.7\text{--}0.9$ for creative tasks and $\tau \approx 0.1\text{--}0.3$ for factual tasks.

9.4 False. Beam search maintains the top- k partial sequences and expands greedily—it is a heuristic best-first search, not an exhaustive search. It is not guaranteed to find the globally highest-probability sequence; in practice it tends to find overly conservative, repetitive outputs.

9.5 Hallucination is confident generation of content that is false or unsupported by context. Factual hallucination: invented facts (non-existent citations, wrong historical dates). Faithfulness hallucination: contradicting information explicitly present in the provided context. The autoregressive training objective rewards next-token prediction correctness, not factual accuracy: truth is not represented in the training signal, so the model has no direct incentive to be accurate.

9.6 With CoT, the model generates intermediate reasoning tokens that become part of the context $x_{<t}$ for subsequent tokens. Each reasoning step can use preceding steps as working memory, allowing multi-step inference that would exceed the model’s effective context compression if attempted in a single token. The mechanism relies on the model having been trained on text that includes step-by-step reasoning, so the CoT prompt activates the corresponding continuation distribution.

9.7 Zero-shot: task specified by instruction alone; relies on general pre-trained knowledge and instruction-following ability. Few-shot: 2–10 input–output examples prepended in the prompt; the model infers the task from the pattern; relies on in-context learning. In-context learning (general): any context—examples, demonstrations, instructions—adapts the model’s continuation without weight updates; relies on the model having seen similar patterns during pretraining.

9.8 BPE starts with individual characters and iteratively merges the most frequent adjacent pair in the corpus into a new compound symbol, until the vocabulary reaches a target size. The result is variable-length subwords: common words become single tokens; rare words decompose into shorter pieces. This avoids the out-of-vocabulary problem of word-level tokenisation while producing shorter sequences than character-level tokenisation.

9.9 B.

9.10 Temperature: divides logits by τ before softmax. $\tau < 1$ sharpens the distribution (more coherent, more deterministic); $\tau > 1$ flattens it (more diverse, less coherent). It does not constrain which tokens are eligible. Top- k : restricts sampling to the k most probable tokens, eliminating the long tail. The problem is that k is fixed regardless of distribution shape—sometimes 10 tokens are natural candidates, sometimes 1000 are. Nucleus (top- p): selects the smallest set of tokens whose cumulative probability reaches p (≈ 0.9 – 0.95). The set adapts: small when the model is confident (one token has near- p probability), large when uncertain. This makes the diversity–coherence balance input-dependent, which is why it is the current default. In practice, top- p and temperature are combined: temperature shapes the distribution first; top- p then cuts the remaining tail.

Section 10 — Modern Image Generation

10.1 B.

10.2 Rectified flow interpolates linearly: $x_t = (1 - t)x_0 + tx_1$, $x_1 \sim \mathcal{N}(0, I)$. The target velocity for this interpolant is $u_t = x_1 - x_0$, a constant vector per sample pair. Training loss: $\mathbb{E}\|v_\theta(x_t, t) - (x_1 - x_0)\|^2$. DDPM paths are curved because the stochastic forward process introduces variance at each step, requiring many small corrective integration steps. Rectified flow paths are straight lines, so the ODE can be integrated with larger steps (or fewer steps with an Euler solver) without significant approximation error.

10.3 True. DDPM corresponds to choosing a stochastic probability path with a specific variance schedule within the flow matching framework. Choosing the DDPM variance schedule as the probability path recovers the DDPM training objective; choosing a linear interpolant recovers rectified flow. Flow matching is strictly more general.

10.4 The consistency property requires $f_\theta(x_t, t) = x_0$ for every t along a given diffusion trajectory. One-step generation: start from $x_T \sim \mathcal{N}(0, I)$ and apply $f_\theta(x_T, T)$ directly to obtain x_0 . Consistency Training (CT) trains from scratch with a self-consistency loss between adjacent trajectory points: $f_\theta(x_t, t)$ must match $f_\theta(x_{t'}, t')$ for nearby t, t' , using a stop-gradient target network. Consistency Distillation (CD) uses a pre-trained multi-step diffusion model as teacher, training f_θ to match the teacher’s two-step output in one step; this produces higher quality than CT at the same step count.

10.5 B.

10.6 DreamBooth fine-tunes all U-Net weights on 3–5 reference images of a specific subject, binding the subject to a rare identifier token (e.g., “a [V] dog”). Without additional constraints, fine-tuning causes catastrophic forgetting: the model becomes highly specialised and loses the ability to generate other subjects or backgrounds. The class-preservation loss simultaneously fine-tunes on synthetic images from the original model for the same broad class (e.g., “a dog”), penalising deviation from the prior generation and preserving the model’s general capability.

10.7 γ (scale) and β (shift); α ; zero; identity.

10.8 ADD combines two losses: (1) a distillation loss matching the student’s 1-step output to the multi-step teacher’s output distribution; (2) a GAN discriminator loss penalising the student when its outputs look perceptually unrealistic compared to real images. Standard distillation alone produces blurry averaged outputs because the student minimises a distribution-level loss that rewards the “average” of multiple plausible outputs. The discriminator provides a direct perceptual feedback signal that penalises this averaging and rewards individual-sample realism.

10.9

	SD3 / FLUX.1	DALL-E 2
Architecture	MMDiT (transformer on latent patches, two token streams)	CLIP prior + cascaded U-Net diffusion decoder
Training objective	Rectified flow	DDPM
Text encoder	CLIP-L + CLIP-G + T5-XXL (concatenated)	CLIP text encoder only
Conditioning	Joint attention in MMDiT (text and image tokens attend mutually)	Cross-attention between U-Net features and CLIP embeddings

10.10 A standard DiT block uses one-directional cross-attention: image patch tokens attend over text token embeddings (separate Q from image, K and V from text). MMDiT maintains two separate token

streams—one for image patches, one for text—each with its own Q, K, V projections. Inside each block, all tokens form a single sequence for the attention operation, so every image token attends to every text token and vice versa. This bidirectional full joint attention in every layer gives the text conditioning deeper influence on image generation, which substantially improves text rendering and compositional accuracy.

10.11

	Autoregressive (VAR, MAR, Emu3)	Diffusion (DDPM, LDM, FLUX)
Speed	VAR $\sim 10\times$ faster than DiT at matched quality (parallel per scale level)	Slow (20–50 steps); 1-step via distillation recovers speed
Architectural unification	Native: same transformer, vocabulary, training loop as LLMs	Separate architecture; text via cross-attention adapters
Quality on benchmarks	VAR: 1.80 FID on ImageNet 256	DiT-XL/2: 2.27 FID; LDM-based leads on T2I benchmarks
Key enabler	Better visual tokenisers (MAGVIT-v2, FSQ) + next-scale prediction	Latent diffusion made high-resolution generation tractable

10.12 VAE at 256×256 : the encoder–decoder architecture scales (add layers), but the MSE reconstruction loss amplifies blurriness at high resolution since the model spreads probability mass over many equally plausible high-frequency pixel configurations. A perceptual loss and/or shifting to LDM (encode to compact latent first, diffuse there) would be necessary. DCGAN at 256×256 : training instability worsens severely because the discriminator can exploit high-frequency artifacts to identify fakes; generating sharp 256×256 images in a single pass is extremely hard. The standard fixes are progressive growing (ProGAN) or the StyleGAN architecture with its mapping network and style injection. DDPM at 256×256 : direct pixel-space diffusion is prohibitively expensive—U-Net self-attention scales quadratically with spatial dimension ($256^2 = 65,536$ positions). The necessary change is Latent Diffusion: encode $256 \times 256 \times 3$ to roughly $32 \times 32 \times 4$ with a frozen VAE ($\approx 64\times$ compression), then run DDPM in that compact latent. Most benefit: DDPM/LDM benefits most from higher resolution. At 32×32 the quality differences between models are compressed; at 256×256 DDPM’s superior mode coverage and fine-detail generation pull decisively away from VAE and GAN. Would you replace architectures? VAE $\rightarrow$ VQ-VAE-2 for sharper textures; DCGAN $\rightarrow$ StyleGAN2 for stable training; plain DDPM $\rightarrow$ LDM is not a replacement but an essential wrapper. A defensible decision would be to drop GANs entirely at this scale, using LDM as the primary model and LoRA for efficient fine-tuning.